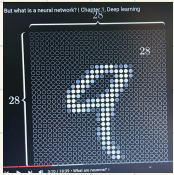


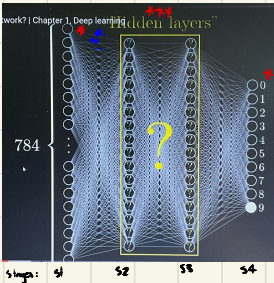
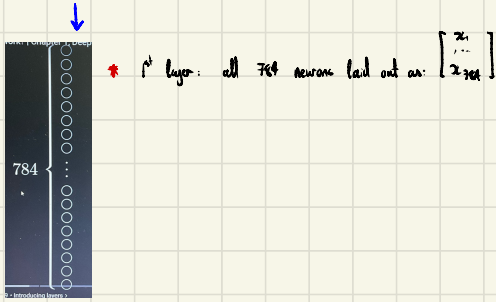
- ①: i) All recognizable as 3
by my brain regardless
of specific values of each pixel.
ii) something internally resolves this

How?



- 2) i) What are neurons?
e: as: thing that holds a number $\in [0,1]$

- 28x28 neuron box
- $\bullet \leftrightarrow$ grayscale value



- ②: "A sort of goal":



- 8-10 little edges



- long + line



- highlight 9

- ③: Let's focus on one stage 2 (<3) Neuron:

- i) - we want neuron g_k to "fire" (be close to 1)
if this image has an edge (as white next to black)
in this region

- ii) what adjustable parameter should the network have

- e: - assign a weight to each
connection between g_k & $\begin{bmatrix} x_1 \\ \vdots \\ x_{784} \end{bmatrix}$
• $w_k \in \mathbb{R}$
• $w_1, w_2, \dots, w_{784}$

- iii) instead of e: the $z_n \in (28 \times 28)$ matrix:

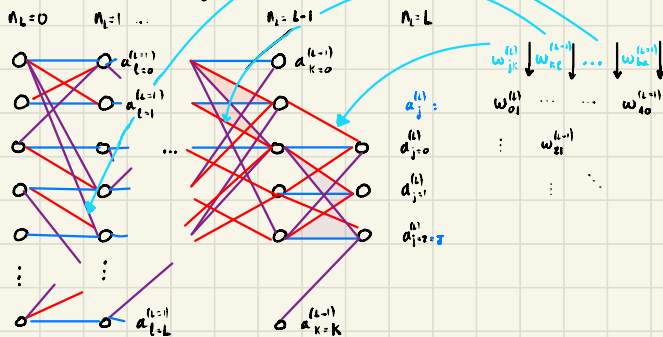
- e: $(z_n \cdot w_n) \in (28 \times 28)$ matrix:
■ $\Rightarrow w_n < 0$ ■ $\Rightarrow w_n > 0$
■ $\Rightarrow w_n < 0$, ■ $\Rightarrow w_n > 0$

- iv) let's make $w_n = 0 \forall w_n$ st $\leftrightarrow z_n$ is far away from

$\Rightarrow z = z_{\text{neighb}} - w_{\text{neighb}}$



Neural network indexing:



Cost function:

assume: i) cost function = $C \propto \text{avg}(C_z) = \frac{1}{n} \sum C_z$

(z = individual training example, eg. $x=42 \leftrightarrow [2, 2] \leftrightarrow \begin{pmatrix} 0 \\ 1 \end{pmatrix}$)

ii) $C = C(a^{(L)})$ (eg. $y_j = \frac{1}{2} \sum_k z_k^{(L-1)} (y_j - a_j^{(L)})^2 \leftrightarrow \frac{1}{2} \sum_k (y_j - a_j^{(L)})^2 = C$)

iii) ** drop C_z for now since linear, for now: $C_z \leftrightarrow C$

$$q: g: S = \{2, 3\}, \quad [(1,1)=1], (c_j, z_j) = \begin{pmatrix} 2 \\ 3 \end{pmatrix} \leftrightarrow [2, 2, 1]]$$

what values do:

$$i) z = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

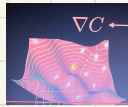
$$iii) w_{jk}^{(L)} = \begin{pmatrix} w_{jk}^{(L-1)} & \dots & w_{jk}^{(L-1)} \\ \vdots & \ddots & \vdots \\ w_{jk}^{(L-1)} & \dots & w_{jk}^{(L-1)} \end{pmatrix}, w_{kL}^{(L-1)}, \dots, w_{bL}^{(L-1)}$$

need to take to minimize the cost of C ?

ans: C_z = cost of 1 training example

assume $\forall$ random initialized values: $b = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, w_{jk} = \begin{pmatrix} 2 & -1 \\ 1 & 1 \end{pmatrix}, w_{bL}$

g : 3 parameters, $\begin{pmatrix} a \\ b \\ c \end{pmatrix}$

$\Rightarrow$  vector calculus produces
vector of steepest incline
geom $\begin{pmatrix} a \\ b \\ c \end{pmatrix} \rightarrow \nabla L(\theta)$

$\therefore$ in our parameter space:

$\Rightarrow -\nabla C(\text{params}) \Rightarrow$ minimization

$$\therefore \nabla C = \frac{1}{n} \sum_{z=0}^{B-1} \left(\frac{\partial C_z}{\partial w} \right) C_z$$

$\theta_j = \frac{\partial C}{\partial \theta_j}(\theta_j)$ (BP1)
This is a very natural expression. The first term on the right, $\partial C / \partial \theta_j$, just measures how fast the cost is changing as a function of the j -th output activation θ_j . For example, C doesn't depend much on a particular output neuron, θ_j , then $\partial C / \partial \theta_j$ will be small, which is what we'd expect. The second term on the right, $\partial C / \partial \theta_j$, measures how fast the activation function is changing at θ_j .

b) c : "error" = $\delta_j^L = \partial C_z / \partial z_j^L$

$$\delta_j^L = \frac{\partial C_z}{\partial z_j^L} = \frac{\partial C_z}{\partial a_j^L} \cdot \frac{\partial a_j^L}{\partial z_j^L} = (\partial a_j^L / \partial C_z) \sigma'(z_j^L) \quad (\text{BP1})$$

$$g: C_z = \frac{1}{2} \sum_j (y_j - a_j^L)^2 \quad (i)$$

$$\Rightarrow \partial a_j^L C_z = (a_j^L - y_j)$$

BP1 is a component-wise expression for vector $\delta^{(L)}$

$\therefore$ in matrix notation

$$(\text{BP1}) \leftrightarrow \delta^L = \nabla_a (z \otimes \sigma'(z^L)) = (a^L - y) \otimes \sigma'(z^L)$$

$$(\theta = [2] \otimes [2] = [1, 2, 3] = [8])$$

$$ii) \delta^L = [(w_{jk}^{(L-1)})^T \delta_k^{(L-1)}] \otimes \sigma'(z^L) \quad (\text{BP2})$$

this process is recursive:

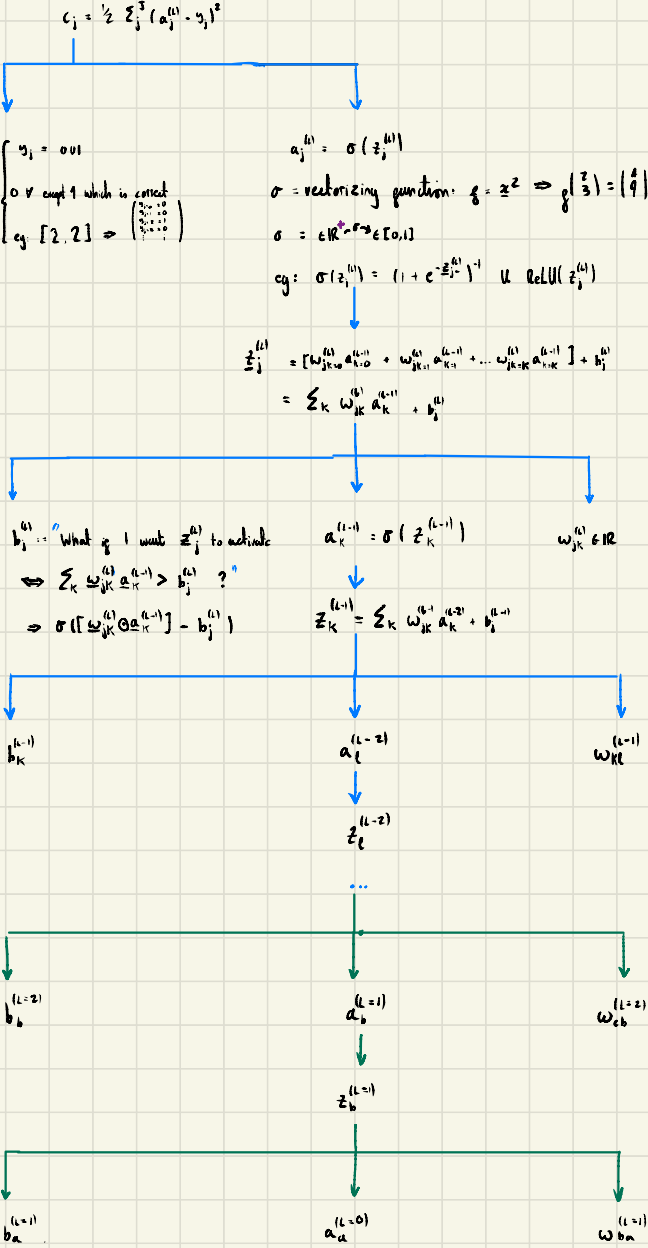
$$g: (\partial a_j^L / \partial C_z) \sigma'(z_j^L)$$

we can calculate $\forall \delta^{(L)}, \delta^{(L-1)}, \dots, \delta^{(1)}$

$$\frac{\partial C_z}{\partial b_j^{(L)}} = \frac{\partial C_z}{\partial a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial b_j^{(L)}} = \frac{\partial C_z}{\partial a_j^{(L)}} \cdot \delta_j^L$$

$$g: z_j^{(L)} = a_j^{(L-1)} \cdot w_{jk}^{(L)} + b_j^{(L)} \\ \Rightarrow \frac{\partial z_j^{(L)}}{\partial b_j^{(L)}} = 1$$

* Note that δ^L is the transpose of the weight matrix $w^{(L-1)}$ for the $\delta^{(L-1)}$ term. The general expression we derived for each element δ_j^L of δ^L has two terms: a term that depends on $\delta_j^{(L-1)}$ and a term that depends on $\delta_j^{(L-1)}$. We can think of this as a recursive expression. The general expression for δ^L is a recursive expression. The general expression for δ^L is a recursive expression. The general expression for δ^L is a recursive expression.



$$\therefore \partial b_j^{(l)} c_\alpha = \delta^\alpha$$

$$k \text{ on short hand: } \partial_b c_\alpha = \delta \quad (\text{BP3})$$

(let: δ is evaluated @ same neuron as k)

$$\begin{aligned} \partial w_{jk}^{(l)} c_\alpha &= \frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial c_\alpha}{\partial a_j^{(l)}} \\ &= \partial w_{jk}^{(l)} z_j^{(l)} \cdot \delta_j^{(l)} \end{aligned}$$

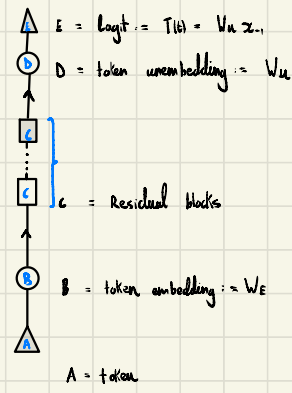
$$g: z_j^{(l)} = a_j^{(l-1)} \cdot w_{jk}^{(l)} + b_j^{(l)} \quad (2)$$

$$\Rightarrow \partial w_{jk}^{(l)} c_\alpha = a_j^{(l-1)} \delta_j^{(l)} \quad (\text{BP4})$$

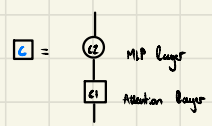
Transformers

Assumptions

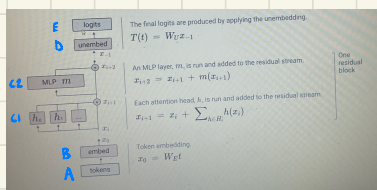
- i) $\mathcal{X}$ MLP (Multi layer processing) [attention layer & MLP typically alternate]
- ii) $\mathcal{X}$ $b_i := \text{bias}$ (see in backpropagation)
- iii) $\mathcal{X}$ layer normalization (some norm($g(x)$) omitted)



token $x_i \leftrightarrow$ word $w_i = z_i = \begin{bmatrix} 0.1 \\ 0.6 \\ -0.5 \end{bmatrix}$ (indicates grid token)
* (actually often 3-4 joint letters in case of longer words for computational efficiency)
for our code (10, 0.8, 24)



$\therefore$ by i) transformer model simplifies to:



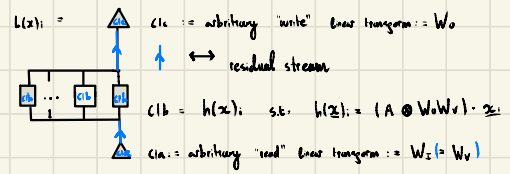
$$C_i = H = \{ H_i \mid i \leq \max(i), i \in \mathbb{Z}^+ \}$$

$$\max(i) = \text{Heads}$$

Each H_i adds its results to "residual stream"

$$\text{residual stream} = \sum_i (\text{previous layers})_i + \text{original embedding}$$

$$= \sum_i L_i + x_0$$



Toy BERT:

- i) assumptions
- ii) $d^k, k = 368$ in distBERT, $K =$ undisclosed for GPT4o
(GPT2 = $368 \leq K \leq 1400$, GPT3 = 12,288)
- iii) heads per layer = 6 in distBERT, = undisclosed for GPT4o
(GPT2 = 20, GPT3 = 96)
- iv) completely linear $\uparrow$ architecture highly unusual
majority contains at least some non-linear activation functions
on their $\uparrow$ or in accessing $\uparrow$
- v) token bandwidth: distBERT = 512
(GPT2 = 1024, GPT3 = 2048, GPT4o = 128,000)

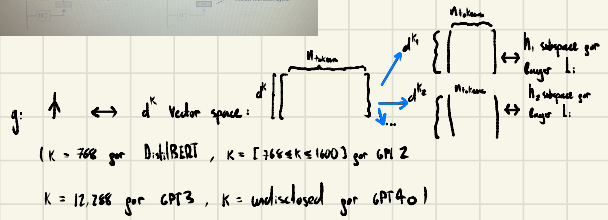
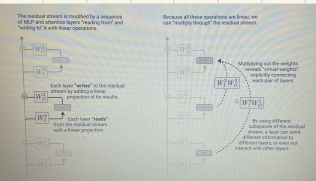
↑ Mathematics:

residual stream. This linear, additive structure of the residual stream has a lot of important implications. One basic consequence is that the residual stream doesn't have a "privileged basis" - we could rotate it by rotating all the matrices interacting with it, without changing model behavior.

additive := $M(X+V) = MX + MV$
 linear := additive + homogeneous
 homogeneous := $A(cX) = cAX$

⇒ suppose I want to know:
 how $L(x)_i$ output affects
 input to $L(x)_i$:

= Mathematically
 useful to keep in mind



$d^K = \mathbb{E} d^{K_1} \cup d^{K_2} \cup \dots \cup d^{K_6}$
 $|K_{A1}| = 128 \cup 64$

in: distilBERT $d_K = \frac{\text{tokens per token}}{\text{heads}} = 76\% = 128$

∴ g: linearity

$h_2(x) \in L_2$ can write to $l \leftrightarrow W_l^T W_0^T h_2(x)$ $h_1(x) \in L_1$

l $h_2(x) \in L_2$ can write to $h_1(x) \in L_1$

simultaneously without the need for $h_2, h_1 \in L_2$ to ever interact

g: $\forall h \in L_i$ information persists in its subspace

unless: $h \in L_{i+1}$ deletes it

⇒ d^K of residual stream can function like memory
 empirically we find: most token embeddings & word embeddings

interact with relatively few dimensions
 $\rightarrow z_0 = \begin{pmatrix} z_0^1 \\ z_0^2 \\ z_0^3 \end{pmatrix}$ $h(x) = \begin{pmatrix} w_1 + c \\ w_2 + c \\ w_3 + c \end{pmatrix}$ (rotational)

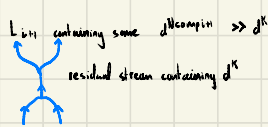
∴ $\forall w \in L_i$ this more efficient to store this information
 in one dimension in the residual stream
 say: $h \in L_1$ & $\forall h \in L_1$ with $d_{h1} - d_{h2} = 6$
 just delete that information & use up the free information space on
 memory:

$h \in L_1 = \begin{bmatrix} w_1 & p_1 & \dots \\ w_2 & p_2 & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$
 $h \in L_1 = \begin{bmatrix} 0 & 0 & \dots \\ 0 & 0 & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$

where we have O_n we use this space for:

"computable dimension" ($= d^{Hcomp}$) (such as neurons & attention head results)

BUT: $d^{Hcomp} \gg d^K$



L_i containing some $d^{Hcomp} \gg d^K$

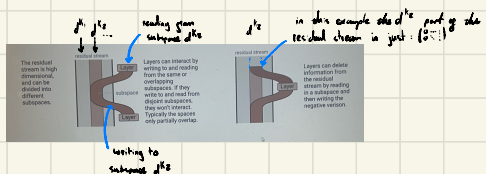
∴ $\exists$ some form of d^{Hcomp} superposition of computational info
 eg: $\begin{pmatrix} w_1 \\ w_2 \\ \vdots \end{pmatrix} \xrightarrow{\text{activation}} \begin{pmatrix} w_1 p_1 + p_1 b_1 \\ \vdots \end{pmatrix}$ (w, p just superposition params $\rightarrow$ GPT)

"bottleneck" := "bottleneck activation tensor"

"bottleneck" := "unusually hard challenging to interpret"

& likely interlinked w MLP information layer

"but some MLP neurons perform memory management roles
 clearing residual stream dimensions set by other layers by
 reading in information & writing out the -ve version."

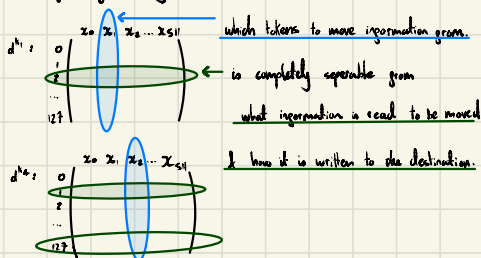


Mathematical Packaging:

q : ↑ locality & additivity

⇒ $\forall h \in H$ are locally independent

⇒ following property:



to illustrate this: c : an attention head $h(x)_i$:

output of $h(x)_i$ broken into 3 steps:

- 1) $V_i = W_V x_i$ compute value vector $x_0, x_1, \dots, x_{S-1}$
 individually: $V_i = \begin{pmatrix} w_{i0}^V & w_{i1}^V & \dots & w_{iS-1}^V \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_{S-1} \end{pmatrix} = \begin{pmatrix} v_{i0} \\ v_{i1} \\ \vdots \\ v_{iS-1} \end{pmatrix}$ (first token in context)
 & packaged: $V = \begin{pmatrix} w_{00}^V & w_{01}^V & \dots & w_{0S-1}^V \\ w_{10}^V & w_{11}^V & \dots & w_{1S-1}^V \\ \vdots & \vdots & \ddots & \vdots \\ w_{S-10}^V & w_{S-11}^V & \dots & w_{S-1S-1}^V \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_{S-1} \end{pmatrix} = \begin{pmatrix} v_{00} & v_{01} & \dots & v_{0S-1} \\ v_{10} & v_{11} & \dots & v_{1S-1} \\ \vdots & \vdots & \ddots & \vdots \\ v_{S-10} & v_{S-11} & \dots & v_{S-1S-1} \end{pmatrix}$

$$2) Q_i = \sum_j A_{i,j} V_j = A_{i,0} V_0 + A_{i,1} V_1 + \dots + A_{i,S-1} V_{S-1}$$

$$V_j = W_V x_j \quad (x_j = (v_0, v_1, v_2) \text{ V } \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} (?))$$

$$A_{(i,j)} = \begin{pmatrix} a_{00} & a_{01} & \dots & a_{0S-1} \\ a_{10} & a_{11} & \dots & a_{1S-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{S-10} & a_{S-11} & \dots & a_{S-1S-1} \end{pmatrix}$$

individually:

$$\& \text{ packaged: } Q = AV = \begin{pmatrix} a_{00} & a_{01} & \dots & a_{0S-1} \\ a_{10} & a_{11} & \dots & a_{1S-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{S-10} & a_{S-11} & \dots & a_{S-1S-1} \end{pmatrix} \begin{pmatrix} v_{00} & v_{01} & \dots & v_{0S-1} \\ v_{10} & v_{11} & \dots & v_{1S-1} \\ \vdots & \vdots & \ddots & \vdots \\ v_{S-10} & v_{S-11} & \dots & v_{S-1S-1} \end{pmatrix}$$

$$3) h(x)_i = W_Q Q_i = W_Q (A_{i,0} V_0 + A_{i,1} V_1 + \dots) = \begin{pmatrix} w_{i0}^Q & w_{i1}^Q & \dots & w_{iS-1}^Q \end{pmatrix} \begin{pmatrix} v_{00} & v_{01} & \dots & v_{0S-1} \\ v_{10} & v_{11} & \dots & v_{1S-1} \\ \vdots & \vdots & \ddots & \vdots \\ v_{S-10} & v_{S-11} & \dots & v_{S-1S-1} \end{pmatrix}$$

$$\leftrightarrow h(x) = (A \otimes W_Q W_V) \cdot x$$

in summary: A mixes across tokens

& $W_Q W_V$ act on each vector independently

$$A := \text{softmax} \left(\frac{QK^T}{\sqrt{d_K}} \right) V$$

$\sqrt{d_K}$: normalization const.

$$Q := W_Q x, \quad q_i = W_Q x_i$$

locally $\leftrightarrow$ query

$$K := W_K x, \quad k_i = W_K x_i$$

locally $\leftarrow$ response

V : temperature

$\therefore$ when they match $\Rightarrow a_{i,j} = \text{large}$

$$\therefore A = \text{softmax} \left(\frac{x^T W_Q^T W_K x}{\sqrt{d_K}} \right)$$

$$g = W_Q \cdot W_K$$